

Taking (good) processing-related decisions: patterns, shuffle, skew, and evidence

The focus of this practical assignment is:

- Decompose analytical workloads into a small processing grammar (filter, project, aggregate, join, window)
- Understand the mechanics behind distributed processing: shuffle, skew bottleneck, spills
- Apply evidence-driven optimization using EXPLAIN / EXPLAIN ANALYZE + Trino UI stats
- Reason about materialization vs query-on-read (when to store results vs compute at query time)
- Apply a practical salting pattern to mitigate skew (and understand its trade-offs)

Scenario

You are building a small analytics “product” for an e-commerce platform. Data arrives as events (page views, add-to-cart, purchases). You also have a customers dimension (country, plan).

Your job is to produce a small set of analytics outputs and, more importantly, to justify the design and optimization choices using evidence:

- What did the engine do (plan)?
- How much data was scanned (bytes/rows)?
- Where did time go (stage breakdown)?
- Did we hit skew or row explosion?
- Should we materialize a result or compute it on read?

You can run queries the same way and using the same Stack as in the previous class (Trino CLI, Trino UI, or SQLTools in VSCode). The important part is that for each relevant query you check:

- EXPLAIN (structure: joins, aggregates, exchanges)
- EXPLAIN ANALYZE (runtime: rows, CPU, blocked time, etc.)
- Trino UI Query Details (processed input bytes/rows, stage stats, memory, spilled bytes if present)

This is exactly the workflow introduced previously: write SQL > run > observe > compare.

1. Create a baseline schema and datasets

Create a new schema for this class as follows.

```
CREATE SCHEMA IF NOT EXISTS iceberg.tead_query_opt  
WITH (location = 's3a://warehouse/tead_query_opt/');
```

```
USE iceberg.tead_query_opt;  
SHOW TABLES;
```

Create a customers dimension. This table will be intentionally small so it is a candidate for broadcast joins.

```
CREATE TABLE customers  
WITH (format = 'PARQUET')  
AS  
WITH  
a AS (SELECT x FROM UNNEST(sequence(1, 400)) t(x)),      -- 400  
b AS (SELECT y FROM UNNEST(sequence(1, 500)) t(y))        -- 500  
SELECT  
  (x - 1) * 500 + y AS customer_id,                        -- 1..200000  
  element_at(ARRAY['PT','ES','FR','DE','US','BR'], 1 + (((x - 1) * 500  
+ y) % 6)) AS country,  
  element_at(ARRAY['free','basic','pro'], 1 + (((x - 1) * 500 + y) % 3))  
AS plan,  
  date_add('day', -(((x - 1) * 500 + y) % 3650), current_date) AS  
signup_date  
FROM a  
CROSS JOIN b;
```

Check result:

```
SELECT count(*) FROM customers;  
SELECT * FROM customers LIMIT 20;  
SELECT min(customer_id), max(customer_id) FROM customers;
```

Create an events fact table. This will be a fairly large table, with a few million rows, created by crossing sequence. Large size is to have enough volume to make plans and stats meaningful.

```
CREATE TABLE events  
WITH (format = 'PARQUET')  
AS  
WITH  
a AS (SELECT x FROM UNNEST(sequence(1, 2500)) t(x)),      -- 2.5k  
b AS (SELECT y FROM UNNEST(sequence(1, 2000)) t(y)),        -- 2k -> 5M  
rows after cross join  
base AS (  
  SELECT (x * 2000 + y) AS event_id  
  FROM a CROSS JOIN b  
)  
SELECT
```

```
event_id,
-- pseudo-event time spread over ~60 days
date_add('day', -(event_id % 60), current_date) AS event_date,
-- event types with a controlled distribution
CASE
  WHEN (event_id % 100) < 70 THEN 'page_view'
  WHEN (event_id % 100) < 90 THEN 'add_to_cart'
  WHEN (event_id % 100) < 95 THEN 'checkout'
  ELSE 'purchase'
END AS event_type,
-- some nulls to simulate imperfect tracking
CASE WHEN (event_id % 20) = 0 THEN NULL ELSE 1 + (event_id % 200000)
END AS customer_id,
element_at(ARRAY['web','ios','android'], 1 + (event_id % 3)) AS source,
-- only meaningful for purchase
CASE WHEN (event_id % 100) >= 98 THEN (10 + (event_id % 200)) * 1.0
ELSE NULL END AS order_total_eur
FROM base;
```

Sanity check:

```
SELECT count(*) FROM events;
SELECT event_type, count(*) FROM events GROUP BY 1 ORDER BY 2 DESC;
SELECT min(event_date), max(event_date) FROM events;
```

2. Baseline: observe a full scan

Start by running a query that cannot be pruned much (without filters):

```
SELECT count(*) FROM events;
```

Note processed input bytes on Trino UI. You might, in some cases, see something like:

Resource Utilization Summary	
CPU Time	230.00ms
Planning CPU Time	60.47ms
Scheduled Time	283.00ms
Input Rows	5.00M
Input Data	0B
Physical Input Rows	5.00M
Physical Input Data	0B
Physical Input Read Time	0.00ns

This means Trino was able to answer based only on the Iceberg metadata manifest/file stats like `record_count`, without scanning Parquet files.

We can “force” a real full scan:

```
SELECT sum(coalesce(order_total_eur, 0))
FROM events;
```

Now you might see something like:

Resource Utilization Summary	
CPU Time	188.00ms
Planning CPU Time	43.24ms
Scheduled Time	198.00ms
Input Rows	5.00M
Input Data	5.15MB
Physical Input Rows	5.00M
Physical Input Data	1.33MB
Physical Input Read Time	9.00ms

3. Grammar pattern: Filter early + Project early (evidence-driven)

Consider the following business question: “We need to calculate the daily revenue by country for the last 7 days”. Consider both options below. If there are no purchases in the last 7 days (purchases make up only 5% of the events, feel free to increase the timespan, or repeat the generation of data increasing the probability of purchases)

Option A:

```
WITH joined_wide AS (
  SELECT
    e.*,           -- <-- intentionally wide
    c.*           -- <-- intentionally wide
  FROM events e
  JOIN customers c
    ON e.customer_id = c.customer_id
)
SELECT
  event_date,
  country,
  sum(order_total_eur) AS revenue
FROM joined_wide
```

```
WHERE event_type = 'purchase'
  AND event_date >= date_add('day', -21, (SELECT max(event_date) FROM
events))
GROUP BY 1,2
ORDER BY 1,2
```

Option B:

```
WITH purchases_last_7d AS (
  SELECT event_date, customer_id, order_total_eur
  FROM events
  WHERE event_type = 'purchase'
    AND event_date >= date_add('day', -7, current_date)
    AND customer_id IS NOT NULL
)
SELECT
  p.event_date,
  c.country,
  sum(p.order_total_eur) AS revenue
FROM purchases_last_7d p
JOIN customers c ON p.customer_id = c.customer_id
GROUP BY 1,2;
```

Run both queries and take notes:

- Which plan has fewer heavy operators before the join?
- How do processed input bytes/rows change?
- Do you see fewer stages / a smaller exchange?

4. Mechanics: Join explosion (semantics and cost)

Let us now consider the issues related to grain and keys discipline, by seeing how a join can become both wrong and expensive.

Start by creating a “bad” dimension with duplicated keys:

```
CREATE TABLE customers_bad
WITH (format = 'PARQUET')
AS
SELECT * FROM customers
UNION ALL
SELECT * FROM customers WHERE (customer_id % 10) = 0;
```

Check:

```
SELECT count(*) FROM customers;
SELECT count(*) FROM customers_bad;

-- How many customer_ids are duplicated?
SELECT count(*) AS duplicated_keys
```

```
FROM (  
  SELECT customer_id  
  FROM customers_bad  
  GROUP BY 1  
  HAVING count(*) > 1  
);
```

Now run the queries below, which are similar, but run on both tables (good and bad).

```
SELECT sum(e.order_total_eur) AS revenue  
FROM events e  
JOIN customers c ON e.customer_id = c.customer_id  
WHERE e.event_type = 'purchase'  
  AND e.event_date >= date_add('day', -30, current_date);  
  
SELECT sum(e.order_total_eur) AS revenue  
FROM events e  
JOIN customers_bad c ON e.customer_id = c.customer_id  
WHERE e.event_type = 'purchase'  
  AND e.event_date >= date_add('day', -30, current_date);
```

Note that:

- Revenue differs (it is inflated in the “bad” case)
- Runtime and/or processed rows may increase (check if the original number of rows read matches the known number of clients)
- You may see a bigger join output (row multiplication)

In real life, this would be a contract/modeling problem, that then becomes a cost/performance problem. Moreover, business might take bad decisions based on this.

5. Mechanics: Shuffle and GROUP BY

Compare low-cardinality vs. high-cardinality group by, by running the queries below.

Low cardinality:

```
SELECT country, count(*)  
FROM customers  
GROUP BY 1;
```

Higher cardinality:

```
SELECT customer_id, count(*)  
FROM events  
WHERE customer_id IS NOT NULL  
GROUP BY 1;
```

Compare:

- Stage complexity (notice how Trino does pre-aggregation and only a small number of rows moves between stages)
- Time
- Bytes
- Any signs of heavy exchange

6. Mechanics: Skew bottleneck + Salting

Let's create a skewed events table, where most rows share a hot key.

```
CREATE TABLE events_skew
WITH (format = 'PARQUET')
AS
SELECT
  event_id,
  event_date,
  event_type,
  source,
  order_total_eur,
  CASE
    WHEN (event_id % 10) < 7 THEN 42 -- 70% hot customer_id
    ELSE customer_id
  END AS customer_id
FROM events;
```

Confirm the existence of skew:

```
SELECT customer_id, count(*) AS n
FROM events_skew
GROUP BY 1
ORDER BY n DESC
LIMIT 10;
```

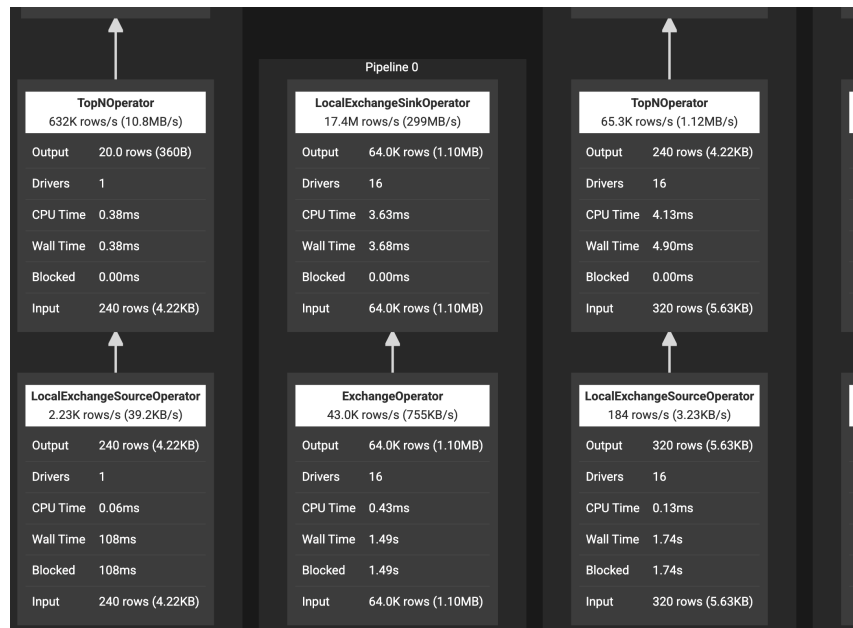
Let us now run a skew-sensitive aggregation

```
SELECT customer_id, count(*) AS n
FROM events_skew
WHERE customer_id IS NOT NULL
GROUP BY 1
ORDER BY n DESC
LIMIT 20;
```

You might observe:

- Long tail / straggler symptoms (might not be visible due to small dataset)
- Runtime increase vs the same query on events

The image below shows how one task is handling much more rows.



Let us now apply salting. To this end, we split the hot key across N buckets to distribute work.

```
WITH salted AS (
  SELECT
    customer_id,
    mod(
      mod(from_big_endian_64(xxhash64(to_utf8(CAST(event_id AS
      varchar)))), 16) + 16,
      16
    ) AS salt_bucket
  FROM events_skew
  WHERE customer_id IS NOT NULL
),
partial AS (
  SELECT customer_id, salt_bucket, count(*) AS cnt
  FROM salted
  GROUP BY 1,2
)
SELECT customer_id, sum(cnt) AS cnt
FROM partial
GROUP BY 1
ORDER BY cnt DESC
LIMIT 20;
```

Compare both plans (there should now be two aggregations) and check if this is more balanced. Results may not be conclusive as we don't have so much data...

4. Materialization vs. Query-on-Read

This step is about productization: do we keep logic as a view (query-on-read) or store a table (materialized gold output)?

Let's start by creating a query-on-read view:

```
CREATE OR REPLACE VIEW v_daily_revenue_by_country AS
WITH purchases AS (
  SELECT event_date, customer_id, order_total_eur
  FROM events
  WHERE event_type = 'purchase'
  AND customer_id IS NOT NULL
)
SELECT
  p.event_date,
  c.country,
  sum(p.order_total_eur) AS revenue
FROM purchases p
JOIN customers c ON p.customer_id = c.customer_id
GROUP BY 1,2;
```

And then query it:

```
SELECT *
FROM v_daily_revenue_by_country
WHERE event_date >= date_add('day', -30, current_date)
ORDER BY event_date, country;
```

Let us now materialize the same result as a table:

```
CREATE TABLE daily_revenue_by_country
WITH (format = 'PARQUET')
AS
SELECT *
FROM v_daily_revenue_by_country;
```

And query it:

```
SELECT *
FROM daily_revenue_by_country
WHERE event_date >= date_add('day', -30, current_date)
ORDER BY event_date, country;
```

Compare the performance of both approaches and think about:

- How does repeated querying differ (runtime, bytes scanned)?
- Which one is more predictable for dashboards?

 ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO	MEI – Mestrado em Engenharia Informática TEAD – Tecnologias Escaláveis para Análise de Dados 2º Semestre – Docente: dcarneiro
---	---

- What new responsibilities appear once you materialize? (refresh cadence, backfills, monitoring)